

Modules

Java Advanced Developer

StayAhead Training; Dec, 2025

About Modules

Modules

- A module is a collection of packages
- Each module has:
 - a list of modules it depends upon
 - a list of services it uses and provides
 - a public API (everything else is internal and inaccessible)
- The Java standard library is composed of modules, e.g. `java.base`

Custom Modules

Modules

- You may choose to deploy your own project as a module/modules
- Module benefits:
 - increased maintainability
 - prevent accidental use of internal APIs
 - easily create runtime images with only as much of the stdlib as needed

Module Declarations: Structure

Modules

```
module $NAME {  
    requires $MODULE; ----- // for each module this one needs  
    exports $PACKAGE; ----- // for each package to be made public  
    opens $PACKAGE; ----- // for each reflect-able package  
    uses $TYPE; ----- // for each service used  
    provides $TYPE with $CLASS; // for each service provided  
}
```

Module Declarations: Example

Modules

```
module java.sql {  
    requires transitive java.logging;  
    requires transitive java.transaction.xa;  
    requires transitive java.xml;  
  
    exports java.sql;  
    exports javax.sql;  
  
    uses java.sql.Driver;  
}
```

Concepts

Modules

Concept	Description
Modular JAR	JAR with module descriptor (<code>module-info.class</code>)
Module path	Like the classpath; module system uses it to locate required modules
Module resolution	Recursive searching of the module path for required modules
Module graph	A hierarchy of modules; initial module at the top, JDK ones at the bottom
Base module	<code>java.base</code> which includes <code>java.lang</code> etc. (actually resolved first!)

Launching Modules

Modules

- Assuming initial module `com.example.app` packaged as `app.jar`
- Assuming required module `com.example.lib` packaged as `lib.jar`
- Assuming Windows (semicolon path separator)

```
java
```

```
--module-path app.jar;lib.jar  
--module com.example.app
```

Reflective Access

Modules

- The `opens` directive permits access to all members at runtime only
- To facilitate integration with reflection-based frameworks

```
module com.example.app {  
    opens com.example.app.entities;  
    ...  
}
```

- Whole modules may be opened, e.g. `open module com.example.app {...}`

Services

Modules

- A service is an abstraction for which there is 0, 1, or n implementations
- Service implementations may be obtained using a `ServiceLoader`
- Necessary when the implementation(s) exist in a separate module
- Classic example: `java.sql.Driver`

Services

Modules

```
module com.example.lib {  
    // this module has the abstraction  
    uses com.example.lib.services.WeatherService;  
}
```

```
module com.example.app {  
    requires com.example.lib;  
    // this module has the implementation  
    provides com.example.lib.services.WeatherService  
        with com.example.app.services.MyWeatherService;  
}
```

Services

Modules

```
// in some class in com.example.lib
```

```
ServiceLoader loader =
```

```
    ServiceLoader.load(WeatherService.class);
```

```
for (WeatherService service : loader) {
```

```
    // TODO
```

```
}
```

```
// or...
```

```
Optional<WeatherService> service = loader.findFirst();
```

The Unnamed & Automatic Modules

Modules

- Q. Can I use modular code in my non-modular lib/app?
- A. Yes - all classpath code is added to the unnamed module
- Q. Can I use non-modular code in my modular lib/app?
- A. Yes - a non-modular JAR on the module-path becomes an automatic module

The Unnamed Module

Modules

- The unnamed module:
 - contains all classes on the classpath (even modular JAR classes)
 - `requires` all module graph modules
 - `exports` and `opens` all its packages
 - `provides` services specified in META-INF/services
- Named modules cannot read the unnamed one

Automatic Modules

Modules

- An automatic module:
 - is named (can be required by other named modules)
 - **requires** all module graph modules
 - **exports** and **opens** all its packages
 - **provides** services specified in META-INF/services
- **An automatic module can read the unnamed one**

Automatic Modules

Modules

- An automatic module is named either by:
 - the `Automatic-Module-Name` header in the JAR's manifest (preferred)
 - the JAR filename with the following caveats:
 - the version part is ignored
 - non alphanumeric characters are replaced by dots
 - e.g. `my-lib-1.3.0.jar` --> `my.lib`

Building Modules

Modules

// compile

```
javac --module-path $DEPS -d $CLASS_DIR $SOURCES
```

// package

```
jar --create --file $JAR --main-class $MAIN_CLASS $CLASSES
```

// execute (assuming Windows)

```
java --module-path $JAR;$DEPS --module $MODULE
```

Building Modules

Modules

Placeholder	Description
\$DEPS	List of dependencies, usually JARs, separated by : (UNIX) or ; (Win)
\$CLASS_DIR	The directory where .class files will be written
\$SOURCES	List of .java files including module-info.java
\$JAR	The path of the JAR file
\$CLASSES	List of .class files created during compilation
\$MAIN_CLASS	Fully qualified name of the main class
\$MODULE	Initial module name

Creating Runtime & Application Images

Modules

- The `jlink` command provides for linking modules into a runtime image
- The image may/may not include application modules
- Two required options:
 1. `--add-modules` (to specify the initial module)
 2. `--output` (the output directory)
- The `--module-path` option is needed when including application modules

Creating Runtime & Application Images

Modules

- Assuming application modules JARs named `app.jar` and `lib.jar`
- Assuming initial module named `com.example.app`
- Assuming Windows

`jlink`

```
--module-path app.jar;lib.jar  
--add-modules com.example.app  
--output app-image
```

Creating Runtime & Application Images

Modules

- jlink performs no service binding when creating an image
- Service provider modules may be included via:
 - `--add-modules` (manual)
 - `--bind-services` (automatic)
- Create image for other OS by adding the JDK's `jmods` dir. to the `module-path`

Launching Runtime & Application Images

Modules

- To launch an application from the image:

```
// assuming image directory named app-image  
// assuming initial module named com.example.app  
app-image/bin/java --module com.example.app
```

- One may include `--module-path` to add modules, e.g. service providers